

Linux & Bash Scripting

Lecture 3 - Variables and user input

Key topics

- Introduction to variables in bash
- Learn how to accept input from the user

Assigning Values to Variables

1. Direct Assignment: `VAR=value`

```
VAR=value
```

No Spaces Allowed:

```
VAR = value # Incorrect! This will be interpreted as a command.
```

Example:

```
count=5  
echo "count: $count"
```

Tip: Use quotes for clarity:

```
echo "It's a good day today" echo "hello \ - backslash and a dash"
```

Common Mistakes

Example of Misunderstanding Variable Scope:

```
name=john ashraf
```

Why does `echo $name` not work?

- Bash interprets `john` as the value and `ashraf` as a command.
- Correct way:

```
name="john ashraf"
```

Example of Scoped Assignment:

```
name=john ls
```

Here, `name` is temporarily assigned for `ls` and discarded after execution.

To make `name` persistent:

```
name=john  
ls  
echo $name
```

Using the read Command for Input

2. Using `read` to Assign Values:

```
read VAR
```

Examples:

```
echo "Enter your name:"  
read name  
echo "Name: $name"
```

Prompt Inline with `-p`:

```
read -p "Enter your age: " age  
echo "Age: $age"
```

Silent Input with `-s` (e.g., for passwords):

```
read -sp "Enter your password: " password  
echo "Password entered"
```

Assigning Variables from Files or Commands

Reading from a File:

```
read hostname < /etc/hostname
```

Example:

```
echo $hostname
```

Alternate Syntax (Legacy):

```
current_directory=`pwd`
```

Preferred Method: `$(pwd)` (modern and easier to nest).

```
echo "Current directory: $current_directory"
```

Assigning Output of a Command:

```
current_directory=$(pwd)
```

Measuring Script Execution Time

Combining Commands and Variables:

```
start=$(date +%s)
sleep 3 # Simulates a delay
end=$(date +%s)
diff=$((end - start))
echo "Time elapsed: $diff seconds"
```

Key Concepts:

- `$(( ... ))`: Performs arithmetic operations.
- `start` and `end`: Store timestamps.

Recap

Key Takeaways:

- Assign variables directly with `VAR=value` (no spaces).
- Use `read` to take input from users:
 - Inline prompts with `-p`.
 - Silent inputs with `-s`.
- Assign output of commands using `$(...)` (preferred).
- Use arithmetic expansion `$(( ... ))` for calculations.
- Always test your scripts to understand variable behavior and scope!